Projekt 9: Bezpieczna detekcja spamu na danych zaszyfrowanych szyfrem Paillierem

Podsumowanie stanu projektu po uruchomieniu eksperymentu (100 SMS). Dokument pomocniczy do prezentacji.

1. Wstęp i cel projektu

- Projekt pokazuje, że klasyfikację SMS-ów można wykonać bez ujawniania serwerowi treści wiadomości ani jawnych wartości cech.
- Zastosowanie: chmurowe filtrowanie wiadomości, w którym dostawca usługi nie powinien czytać korespondencji klientów.
- Pytanie badawcze: czy klasyfikacja Naive Bayes na danych zaszyfrowanych Paillierem daje wyniki porównywalne z klasyfikacją jawną?
- Model zagrożeń: Honest-but-Curious. Serwer wykonuje obliczenia poprawnie, ale może próbować poznać dane klientów.
- Podział ról:
  - **Klient**: zamienia SMS na wektor TF-IDF, generuje klucze, szyfruje cechy, odszyfrowuje wynik i podejmuje decyzję.
  - **Serwer**: ma wytrenowany model Naive Bayes w postaci jawnych wag, liczy na zaszyfrowanych danych i odsyła zaszyfrowany wynik.
- Klucz prywatny zna tylko klient.

2. Etap I: dane, TF-IDF i model jawny

Dane

- Zbiór SMS Spam Collection z UCI: 5574 wiadomości w formacie tekstowym (etykieta i treść oddzielone tabulatorem).
- Usunięcie duplikatów: z 5574 zostaje 5171 wiadomości (ham 4518, spam 653).
- Sprawdzenie spójności: nie może być wiadomości z dwiema różnymi etykietami.
- Kodowanie *latin-1*, bo zbiór nie jest w UTF-8.

Podział

- 80% trening, 20% test, podział stratyfikowany, ziarno 42.
- Trening: 4136 wiadomości. Test: 1035 wiadomości.
- Test nie jest używany do wyboru parametrów. Służy tylko do końcowej oceny.
- Trening dzielimy dodatkowo na część uczącą i walidacyjną (20%), żeby dobrać alpha i próg.

TF-IDF

- Każda wiadomość zamieniana jest na wektor 300 wartości, po jednej na słowo ze słownika.
- Używamy *sublinear_tf=True*, czyli logarytmicznego skalowania częstości słów. Ogranicza to wpływ słów powtarzających się wiele razy w jednym SMS-ie.
- Słownik i wartości IDF są wyuczone na zbiorze treningowym.
- TF-IDF działa po stronie klienta. Klient ma słownik, a serwer nigdy nie widzi tekstu ani wektora w postaci jawnej.

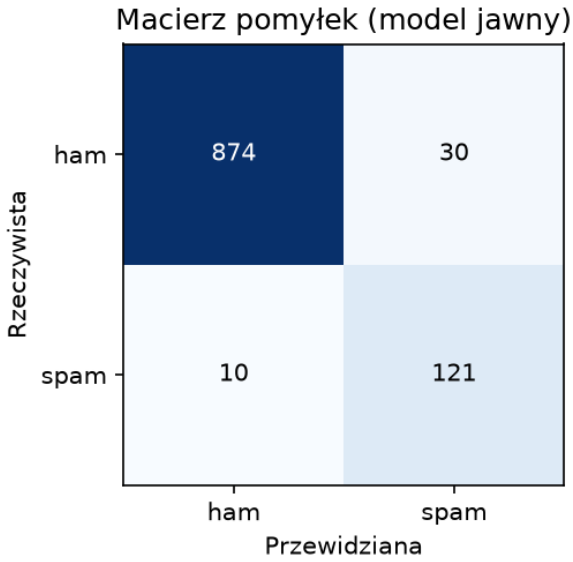
Model Naive Bayes

- Multinomial NB: dla każdej klasy model przechowuje log-prawdopodobieństwo każdego słowa oraz log-prawdopodobieństwo klasy (prior).

- Wynik klasy to suma iloczynów wektora cech i logarytmów prawdopodobieństw, plus prior.
- Decyzja: margines = wynik(spam) − wynik(ham). Jeśli margines ≥ próg, to spam.
- Alpha wygładza prawdopodobieństwa słów, których nie było w treningu. Wybieramy je na walidacji z siatki 0,01–1,0.
- Próg wybieramy na walidacji tak, by recall spamu był ≥ 90%. Spośród progów spełniających ten warunek wybieramy ten z najwyższym F1.

Wynik na zbiorze testowym (model jawny)

- Accuracy 96,14%, precision 80,13%, recall 92,37%, F1 0,858.
- Macierz pomyłek [[874, 30], [10, 121]]: 874 ham poprawnie, 30 ham jako spam, 10 spamów przeoczonych, 121 spamów wykrytych.



Rys. 1. Macierz pomyłek modelu jawnego na zbiorze testowym.

3. Etap II: szyfrowanie Paillierem

Dlaczego Paillier

- Jest częściowo homomorficzny: pozwala dodawać zaszyfrowane wartości i mnożyć je przez liczbę jawną.
- Naive Bayes jest liniowy względem cech: wynik to suma cech pomnożonych przez wagi. Do obliczenia wyniku wystarczy więc dodawanie i mnożenie przez stałą.
- Szyfrowanie jest probabilistyczne, więc ten sam tekst daje różne szyfrogramy.

Klucze

- Klucz Pailliera o długości 2048 bitów, biblioteka *phe* 1.5.0.
- Parametr *n* jest jawny, a klucz prywatny zostaje u klienta.
- Klucz 1024 bity służy tylko do szybkiego testu (*--quick*).

Kodowanie liczb

- Paillier działa na liczbach całkowitych modulo *n*, a cechy TF-IDF są ułamkami.
- Cechy kodowane są przez mnożenie przez 10^6 i zaokrąglenie do liczby całkowitej.
- Wagi modelu kodowane są skalą 10^5, a priory skalą 10^6 × 10^5, żeby wynik miał spójną skalę.
- Ujemne log-prawdopodobieństwa są obsługiwane przez *phe*.

- Błąd kodowania nie może przekroczyć 0,5 / 10^6. Klient sprawdza to po odszyfrowaniu swoich cech.

Szyfrowanie

- Każda z 300 cech szyfrowana jest osobno. Dla jednego SMS-a powstaje 300 szyfrogramów.
- Szyfrogram ma wykładnik 0, więc serwer może dalej operować na nim bez dodatkowych przekształceń.
- Koszt: około 14 ms na jedną cechę, czyli około 4,2 s na SMS.

4. Etap III: klasyfikacja na danych zaszyfrowanych

Co dostaje serwer

- Publiczny klucz (n).
- 300 zaszyfrowanych cech.
- Jawny model: wagi dla ham i spam oraz priory, zakodowane skalami z etapu II.
- Serwer nie dostaje tekstu, wektora TF-IDF w postaci jawnej, klucza prywatnego ani progu decyzji.

Co liczy serwer

- Dla każdej klasy liczy E(prior + Σ x_i · w_i).
- Mnoży każdy szyfrogram przez wagę (liczbę jawną) i dodaje wyniki.
- Na końcu ponownie losuje szyfrogram (rerandomizacja), żeby wynik nie był łatwy do powiązania z wejściem.
- Odsyła do klienta dwa szyfrogramy: wynik dla ham i dla spam.

Co robi klient

- Odszyfrowuje oba wyniki i przelicza je na wartości zmiennoprzecinkowe.
- Liczy margines spam − ham i porównuje z progiem z treningu.
- Podejmuje decyzję i zna jej wynik. Serwer tej decyzji nie zna.

Dlaczego to dobrze działa

- Wynik zaszyfrowany jest matematycznie równy wynikowi jawnemu, z dokładnością wynikającą z kodowania.
- Zmiana kolejności obliczeń nie wpływa na wynik.

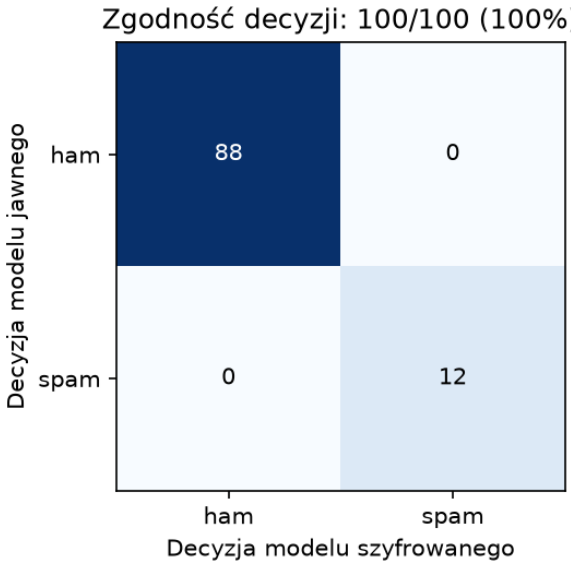
5. Etap IV: testy i porównanie

Próbka

- 100 SMS-ów ze zbioru testowego, stratyfikowana: ham 87, spam 13. Ziarno 42.
- Dla każdego SMS-a liczymy decyzję jawną i szyfrowaną oraz czasy poszczególnych etapów.

Zgodność

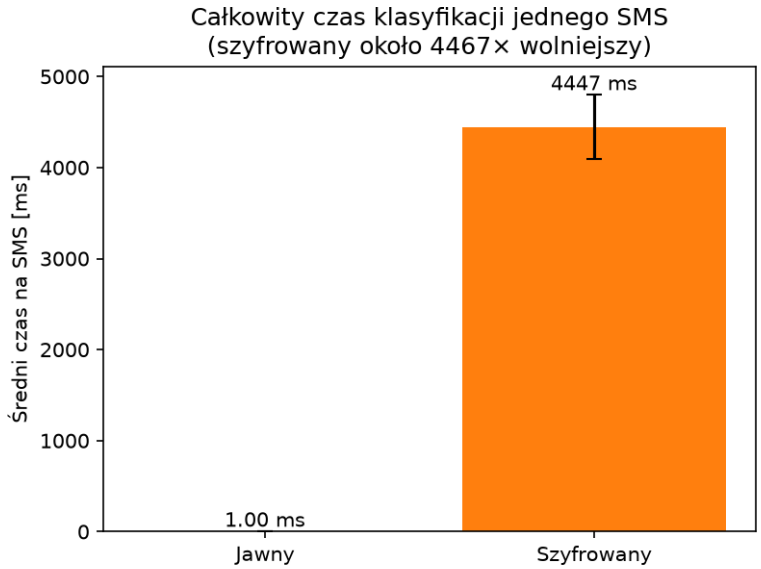
- Zgodność decyzji: 100%, zero rozbieżności.
- Accuracy na próbce: 99% w obu wariantach.



Rys. 2. Zgodność decyzji jawnych i szyfrowanych na próbce 100 SMS-ów.

Czasy (średnio na SMS)

- Wariant jawny: 0,995 ms.
- Wariant szyfrowany: 4447 ms.
  - TF-IDF: 0,548 ms.
  - Szyfrowanie: 4235 ms.
  - Klasyfikacja na serwerze: 203 ms.
  - Odszyfrowanie i decyzja klienta: 8,3 ms.
- Wariant szyfrowany jest około 4500 razy wolniejszy.



Rys. 3. Średni całkowity czas klasyfikacji jednego SMS-a w obu wariantach.

Wnioski z czasów

- Prawie cały koszt to szyfrowanie po stronie klienta, a nie samo obliczenie na serwerze.
- Czas szyfrowania rośnie liniowo z liczbą cech i mocniej z długością klucza.
- Serwer liczy szybciej niż klient szyfruje, bo mnoży przez małe wagi, a nie przez pełne klucze.

6. Decyzje projektowe i ich uzasadnienie

- **Naive Bayes zamiast bardziej złożonego modelu**: liniowość wyniku pozwala użyć Paillier bez kosztownych operacji (np. nieliniowych funkcji).
- **Dokładnie 300 cech**: analiza wrażliwości (10 podziałów danych) pokazuje, że 100 cech obniża precision do ok. 0,45–0,66, a 300 daje ok. 0,79–0,89. Powyżej 300 zysk jest niewielki, a koszt szyfrowania rośnie.
- **Alpha i próg na walidacji**: test nie jest używany do strojenia, więc wyniki na nim są uczciwą oceną.
- **Recall ≥ 90% jako kryterium**: wymóg zlecenia. Dla spamu błąd przeoczenia jest zwykle groźniejszy niż fałszywy alarm.
- **Klucz 2048 bitów**: standardowy poziom bezpieczeństwa dla Paillier. Krótszy klucz byłby szybszy, ale mniej bezpieczny.
- **Próbka 100 SMS-ów**: kompromis między czasem trwania eksperymentu (ok. 7 minut) a liczbą obserwacji.
- **Stałoprzecinkowe kodowanie**: prostsze i deterministyczne niż kodowanie zmiennoprzecinkowe, a błąd jest mierzalny i mały.

7. Specyfikacja techniczna

- System: Windows 10, Python 3.13.2 w *.venv*.
- Biblioteki (wersje z instalacji): *phe* 1.5.0, *gmpy2* 2.3.2, *numpy* 2.5.4, *pandas* 3.1.0, *scikit-learn* 1.9.1, *matplotlib* 3.11.2, *reportlab* 5.1.0.
- Zbiór: *SMSSpamCollection* (lokalnie w katalogu projektu).
- Parametry domyślne (*spam_he/config.py*): ziarno 42, podział 80/20, walidacja 20% treningu, 300 cech, klucz 2048 bitów, skala cech 10^6, skala wag 10^5.
- Uruchomienie: *python run_experiment.py --samples 100* (etapy I–IV, wyniki w katalogu *wyniki*).
- Wyniki: *raport.txt*, *porownanie.csv*, wykresy PNG, *raport.pdf* generowany automatycznie.

8. Wnioski

- Klasyfikacja na danych zaszyfrowanych jest wykonalna i daje ten sam wynik co wersja jawna (100% zgodności na próbce).
- Ceną prywatności jest czas: wariant szyfrowany jest około 4500 razy wolniejszy na SMS.
- Główne wąskie gardło to szyfrowanie po stronie klienta. Serwer jest szybki.
- Hipoteza badawcza została potwierdzona na tej próbce, z zastrzeżeniem małej liczby spamów.
- Wybór modelu liniowego (Naive Bayes) był kluczowy, bo pozwolił użyć Paillier bez przybliżeń.

9. Ograniczenia

- Próbka 100 SMS-ów (13 spamów). Nawet przy 100/100 zgodnych decyzji dolna granica 95% przedziału ufności wynosi ok. 96,4%.
- Pomiar czasu lokalny: bez sieci, bez generowania kluczy i bez transmisji.
- Wagi modelu są jawne na serwerze. Chronimy dane klienta, nie model.
- Klient zna wynik decyzji, więc prywatność wyniku nie jest pełna.
- Zakładamy uczciwy serwer. Nie ma weryfikacji poprawności obliczeń.
- Słownik 300 cech ogranicza skuteczność modelu.

- Zaokrąglenia stałoprzecinkowe mogą zmienić decyzję przy progu.